UNIVERSIDAD TÉCNICA FEDERICO SANTA MARÍA

SEDE CONCEPCIÓN · DEPARTAMENTO DE ELECTRÓNICA E INFORMÁTICA

TALLER DE ADMINISTRACIÓN DE SISTEMAS

Proyecto integrador · Nodo Sur 2026

ENTREGA 1 · INFORME DE DISEÑO Y PLANIFICACIÓN

DATOS DE IDENTIFICACIÓN	
Título del proyecto	Nodo Sur — Plataforma digital resiliente para una red de emprendimientos y experiencias regionales
Equipo / grupo	*6*
Integrantes	*JoseCarlos Nicolas Vidal Quisel, Octavio Vicente Valencia Henriquez, Lukas Alberto Garrido Maldonado.*
Docente	José Antonio Arellano V.
Asignatura	EIN-090B · Taller de Administración de Sistemas
Fecha de entrega	*[10/09/2026]*
Versión del informe	v1.0
Respositorio	https://github.com/LukasGarrido/TAS

Segundo semestre · 2026

Ponderación: 10% de la calificación grupal del proyecto

ÍNDICE DE CONTENIDOS

Actualice esta tabla de contenido en Word: clic derecho > Actualizar campo.

1 INTRODUCCIÓN

Nodo Sur es una red regional que agrupa pequeñas marcas, talleres creativos, productores y operadores de experiencias, quienes actualmente gestionan sus ventas, reservas, inscripciones y atención a clientes mediante las redes sociales, con formularios aislados y mensajería. Actualmente esta modalidad de operación dificulta la centralización de la información, aumenta el riesgo de perdida de datos y genera problemas de disponibilidad de servicios.

El proyecto tiene como fin diseñar e implementar una plataforma digital, la cual sea centralizada, administrable y resiliente, que permita a distintos miembros de Nodo Sur publicar sus productos y experiencias, gestionar pedidos y reservas, ofrecer contenidos y mantener comunicación con sus clientes desde un único dominio, manteniendo la continuidad del servicio web incluso ante la falla de uno de los nodos web.

El alcance de esta primera entrega corresponde al levantamiento de requerimientos, la selección de la aplicación base y el diseño de la arquitectura. Durante el trabajo la arquitectura evolucionó respecto de la propuesta inicial (instalación tradicional con Nginx y PHP-FPM): los servicios pasaron a ejecutarse en contenedores Docker, se incorporó una máquina virtual de monitorización y los archivos compartidos entre backends se resolvieron con un servidor NFS alojado en la VM de la base de datos. Este informe describe la arquitectura vigente.

Como restricción principal, la organización no cuenta con un equipo de desarrollo permanente, por lo que la solución debe basarse en software libre o de bajo costo y ser administrable por personal con capacitación básica. Además, el tráfico presentará picos breves de demanda durante las campañas de difusión, lo que condiciona directamente las decisiones de disponibilidad y escalabilidad.

Las fallas del caso piloto se traducen en pérdidas e interrupciones concretas sobre los activos y flujos de negocio. La siguiente tabla vincula cada riesgo con su consecuencia y con el requisito que lo cubre.

Activo / flujo	Falla	Pérdida o interrupción concreta	Requisito
Base de datos (pedidos, reservas, inscripciones)	Servicio caído o dato perdido	Sin ventas ni reservas mientras dure la falla; pedidos o inscripciones sin registro; cupos sobrevendidos o perdidos	RNF-02, RNF-06
Archivos subidos (servidor NFS)	Servidor NFS caído, divergencia o disco lleno	Imágenes de catálogo no disponibles en uno o ambos backends; imposibilidad de publicar productos	RNF-03
Nodo perimetral (DNS + Nginx)	Caída del host	Sitio completo inaccesible aunque los backends estén sanos	RNF-01, RNF-05

Activo / flujo	Falla	Pérdida o interrupción concreta	Requisito
Backend web (CMS 1 o CMS 2)	Caída de un nodo	Sin impacto si el otro absorbe la demanda; si no la absorbe, degradación o interrupción de ventas durante la campaña	RNF-01
Flujo de reserva con cupos	Escritura duplicada o perdida	Doble reserva o cupo perdido (solicitudes duplicadas del incidente piloto)	RF-01
Respaldos	Backup antiguo o nunca restaurado	Pérdida de semanas de datos y recuperación incierta (respaldo sin probar del incidente piloto)	RNF-06
Credenciales administrativas	Compartidas, expuestas o sin revocación	Acceso indefinido de personas que ya no deberían tenerlo y sin trazabilidad	RNF-04
Monitorización y logs	Ausencia de métricas y alertas	Fallas de horas sin diagnóstico (formulario caído casi dos horas)	RNF-05, RNF-08

1.1 Análisis de Requisitos

ID	Requisito verificable	Prioridad	Umbral de aceptación y prueba
RF-01	CMS funcional para catálogo, pedidos, reservas y administración de contenidos	Alta	Umbral: un flujo completo (producto o reserva a pedido registrado) se ejecuta bajo el dominio único y el registro es localizable en la aplicación y en la BD. Prueba: ejecutar la compra o reserva desde un cliente y consultar la tabla en MariaDB.
RNF-01	Capa web operativa con al menos dos nodos web	Alta	Umbral: con un backend detenido, ≥ 99 % de 100 solicitudes GET consecutivas responden 200 y la conmutación ocurre en ≤ 30 s. Prueba: detener el contenedor de WordPress de un backend, lanzar 100 solicitudes con curl y revisar los logs de Nginx.
RNF-02	La base de datos opera de manera independiente y no expuesta	Alta	Umbral: el puerto 3306 solo responde a 10.33.199.50, .51 (WordPress) y .52 (monitorización); cualquier otro origen es rechazado. Prueba: nc -zv a 10.33.199.53:3306 desde un origen autorizado y desde uno no autorizado.
RNF-03	Persistencia y coherencia del estado compartido	Alta	Umbral: un archivo subido por un backend se sirve con HTTP 200 y el mismo checksum desde el otro en ≤ 5 s. Prueba: subir una imagen y comparar sha256sum de la descarga por ambos backends.

ID	Requisito verificable	Prioridad	Umbral de aceptación y prueba
RNF-04	Administración remota solo por canales seguros y nominativos	Media	Umbral: SSH por clave pública, un usuario por integrante, sin contraseña ni cuentas compartidas; una cuenta se revoca sin afectar a las demás. Prueba: revisar sshd_config y revocar una clave de prueba.
RNF-05	Detección de falla crítica en tiempo acotado	Media	Umbral: alerta de Alertmanager recibida en ≤ 5 min tras detener un backend, sin observar un panel. Prueba: detener un servicio y medir el tiempo hasta la recepción de la alerta en el canal definido.
RNF-06	Continuidad mediante respaldos verificados	Alta	Umbral: retención ≥ 30 días; restauración completa en ≤ 60 min con pérdida de datos ≤ 24 h. Prueba: restaurar en una VM de prueba midiendo tiempos y datos recuperados (Entrega 4).
RNF-07	Publicación bajo un dominio único con HTTPS válido	Alta	Umbral: https://tas-06.arpa responde sin error de certificado y HTTP en el puerto 80 redirige a HTTPS. Prueba: curl -v y openssl s_client desde un cliente.
RNF-08	Trazabilidad y hora consistente en logs	Media	Umbral: desfase entre nodos ≤ 1 s y capacidad de reconstruir un evento entre logs de proxy, web, BD y SSH. Prueba: chronyc tracking en todos los nodos y seguimiento de una solicitud entre logs (consulta en Loki).

2 RECOMENDACIÓN DE LA APLICACIÓN

Criterio	Alternativa A (Joomla)	Alternativa B (Drupal)	Decisión (WordPress)
Licenciamiento	GPL, gratuito	GPL, gratuito	WordPress: GPL, gratuito. Empate en costo entre las tres opciones; no es un factor diferenciador
Comunidad/soporte	Grande (tercer CMS más usado a nivel mundial), pero con ecosistema de extensiones más pequeño que WordPress	Grande pero orientada a desarrolladores y proyectos enterprise/gobierno, menos enfocada en e-commerce	WordPress, por su ecosistema maduro de plugins de e-commerce y reservas

Criterio	Alternativa A (Joomla)	Alternativa B (Drupal)	Decisión (WordPress)
Curva de operación	Media: requiere entender su estructura de artículos, categorías y menús, más rígida que WordPress	Alta: requiere conocimientos técnicos serios de arquitectura de contenidos, módulos y, en ocasiones, Drush/CLI	WordPress, por menor curva para personal sin equipo técnico permanente
Recursos	Pila PHP + base de datos MySQL/MariaDB con requisitos comparables a WordPress	Pila similar; con Commerce y módulos a medida exige más tuning y recursos	WordPress: imagen oficial wordpress:php8.3-apache en contenedor y MariaDB 11. 2 vCPU / 2 GB por backend como base, a validar con prueba de carga
Mantenimiento	Actualizaciones mayores más exigentes y extensiones con ciclos de soporte desiguales	Actualizaciones mayores costosas; requiere personal técnico para mantenerlo	WordPress: parches frecuentes; el mayor riesgo son los plugins de terceros. Política: actualizar primero en pruebas, limitar plugins y deshabilitar la edición de archivos desde el panel. Las imágenes se versionan por etiqueta
Compatibilidad	Compatible con la pila, pero e-commerce y reservas dependen de extensiones poco integradas	Compatible con la pila; el e-commerce exige Drupal Commerce y desarrollo a medida	WordPress: compatible con Docker, MariaDB y NFS; WooCommerce exige versiones mínimas de WordPress y PHP; el plugin de reservas debe validarse con dos nodos
Límites conocidos y de operación	No tiene un módulo de e-commerce/reservas tan maduro como WooCommerce; VirtueMart o HikaShop son menos pulidas	No tiene e-commerce/reservas nativo; requiere Drupal Commerce y desarrollo a medida, con mantenimiento más costoso	WordPress: es stateful (uploads en disco), lo que con dos nodos obliga a resolver la coherencia de archivos; wp-cron se ejecuta por duplicado con dos nodos (se resuelve con cron del sistema en uno solo); admin-ajax carga bajo tráfico alto; sensible a plugins, requiere caché

Se evaluaron tres alternativas para el caso de negocio: WordPress, Joomla y Drupal. La aplicación seleccionada es WordPress junto con WooCommerce, ejecutado en contenedores Docker detrás de Nginx. Joomla se descarta porque, si bien es GPL y tiene una comunidad grande, requeriría sumar extensiones de terceros poco integradas entre sí (como VirtueMart o HikaShop) para cubrir catálogo, cupos y venta directa, agregando fragilidad sin un beneficio real sobre WordPress. Drupal se descarta porque está pensado para sitios complejos con lógica de contenido avanzada, no para una tienda con reservas simple, y exigiría un equipo de desarrollo dedicado que la organización no tiene.

WordPress + WooCommerce cubre en un solo stack el catálogo, los pedidos, la gestión de contenidos y, con un plugin adicional de reservas (a definir; candidatos: Amelia o Bookly), las reservas de talleres con cupos. Los backends usan la imagen oficial de WordPress con Apache y PHP 8.3, lo que simplifica el despliegue y la réplica por contenedor, mientras que Nginx actúa como proxy inverso y balanceador en el nodo perimetral, donde su manejo eficiente de conexiones concurrentes es relevante dado que el caso anticipa picos de tráfico durante campañas de difusión. El riesgo asumido es la dependencia del ecosistema de plugins y de mantenerlos actualizados, que se mitiga con la política de mantenimiento indicada en la tabla. Las versiones mínimas exactas de cada CMS deben confirmarse en su documentación oficial.

3 ARQUITECTURA DE SERVICIOS

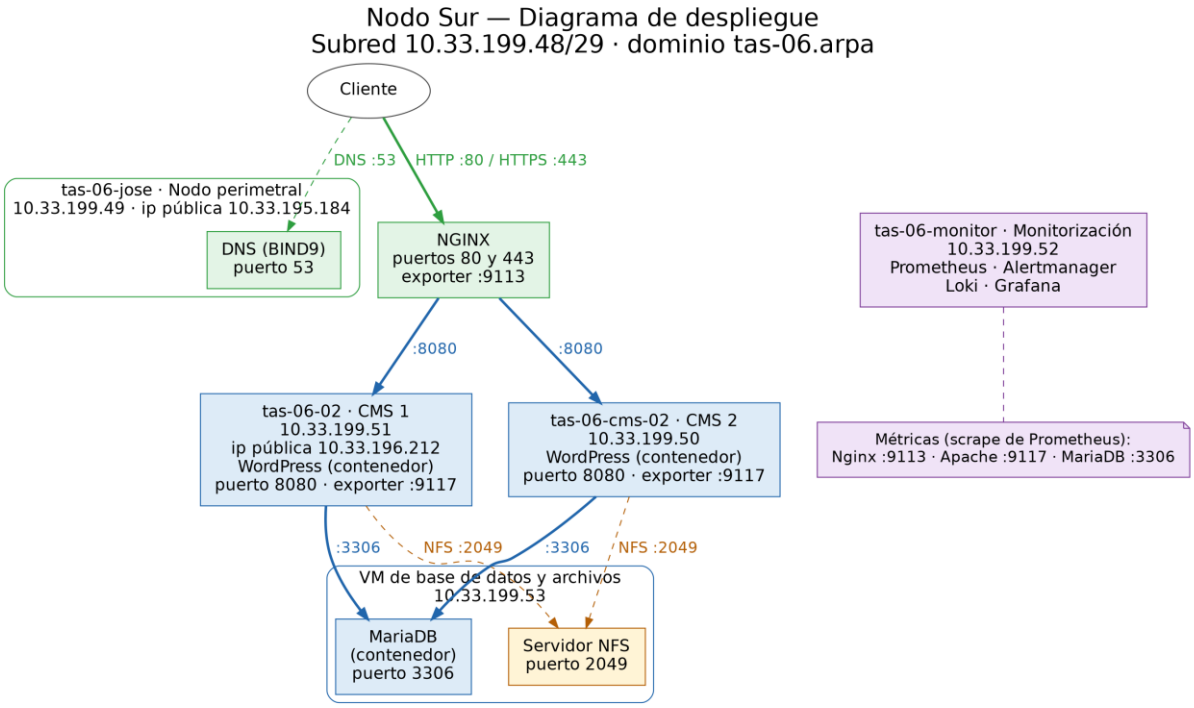


Figura 1. Arquitectura de servicios vigente sobre la subred 10.33.199.48/29. Fuente: elaboración propia.

La arquitectura (Figura 1) se despliega en cinco máquinas virtuales sobre la subred 10.33.199.48/29 y el dominio interno tas-06.arpa, con los servicios ejecutándose en contenedores Docker. El cliente resuelve el dominio en el nodo perimetral (tas-06-jose, 10.33.199.49, IP pública 10.33.195.184), donde DNS (BIND9) y Nginx corren en el mismo host, y accede por los puertos 80 y 443. Nginx distribuye las solicitudes hacia dos backends WordPress equivalentes.

Cada backend (CMS 1 en 10.33.199.51 y CMS 2 en 10.33.199.50) ejecuta WordPress en un contenedor (imagen wordpress:php8.3-apache) que publica el puerto 8080 hacia Nginx. Ambos se conectan por red privada a MariaDB (10.33.199.53, contenedor, puerto 3306), accesible únicamente desde los backends y desde el monitor (10.33.199.52). El uso de Nginx como balanceador en lugar de HAProxy está documentado en el ADR-002.

Coherencia de archivos entre nodos: dos backends WordPress pueden entregar contenidos distintos si no se define dónde vive cada tipo de estado. La VM de la base de datos también actúa como servidor NFS (puerto 2049) y comparte con ambos backends el directorio /srv/nfs/wordpress: wp-config.php, montado desde

/mnt/wp-config, de modo que ambos nodos usan la misma configuración, claves y salts, y por ello las sesiones de inicio de sesión valen en los dos; y wp-content/uploads, montado como volumen NFS. Las sesiones de WooCommerce, los carritos y los transients se guardan en la base de datos, por lo que ya son coherentes. Se eligió alojar el NFS en la VM de la base de datos y no en un nodo propio por simplicidad y por la escasez de direcciones del /29; el costo es que la base de datos y los archivos comparten punto único de falla y compiten por el disco (ADR-003). Los plugins y temas siguen en volúmenes locales de cada backend, por lo que la política es deshabilitar su instalación desde el panel (DISALLOW_FILE_MODS) y desplegarlos desde archivos versionados; esta medida está pendiente de aplicar.

Alcance de los health checks: la versión libre de Nginx solo ofrece detección pasiva (max_fails y fail_timeout con proxy_next_upstream ante error, timeout y respuestas 502, 503 y 504). Esto detecta un servicio caído, una conexión rechazada, timeouts y errores 5xx del backend, pero no detecta contenido distinto entre nodos, respuestas 200 lentas que no superan el timeout ni la caída de la base de datos, que afecta por igual a ambos backends. Además, Nginx no reintenta por defecto solicitudes no idempotentes: un pedido (POST) que falla a mitad de camino no se reenvía al otro backend, lo que evita duplicarlo pero implica que ese cliente ve el error. Como complemento, Prometheus recolecta el estado de Nginx, de Apache y de MariaDB, y se incorporará un script con curl a un endpoint de salud de cada backend, ejecutado por cron y con alerta.

Acceso directo a backends: hoy el puerto 8080 de cada backend se publica en todas sus interfaces y el CMS 1 conserva una IP pública (10.33.196.212), por lo que un cliente podría eludir los controles de Nginx. El control consiste en publicar el puerto solo en la IP privada de cada nodo, restringir con firewall el origen a 10.33.199.49 y retirar la IP pública del CMS 1. Como Docker inserta sus propias reglas de iptables y se salta el firewall del host, la restricción debe aplicarse también en la cadena DOCKER-USER. La aceptación se verifica con dos pruebas: un curl a la IP pública del CMS 1 desde fuera debe fallar y un curl al puerto 8080 de un backend desde un origen distinto de .49 debe ser rechazado.

Monitorización: la VM tas-06-monitor (10.33.199.52) ejecuta Prometheus, Alertmanager, Loki y Grafana en contenedores, con sus puertos publicados solo en 127.0.0.1 y acceso por túnel SSH. Prometheus recolecta métricas de nginx-prometheus-exporter (9113) en el nodo perimetral, de apache-exporter (9117) en cada backend y de mysqld-exporter (9104, en la VM del monitor), que consulta MariaDB por el puerto 3306 con un usuario de solo lectura de métricas. El canal de estadísticas de BIND9 (127.0.0.1:8053) está habilitado, pero aún no tiene exporter. Esta VM mantiene una IP pública temporal (10.33.195.245) que se retirará al terminar la configuración.

Servicio / rol	Ubicación	Puerto o flujo	Dependencias
DNS (BIND9) + Nginx (proxy/balanceador)	Nodo perimetral tas-06-jose: 10.33.199.49 (pública 10.33.195.184)	53 DNS · 80/443 TCP HTTP/HTTPS · 9113 métricas de Nginx	CMS 1, CMS 2
WordPress CMS 1 (contenedor Apache + PHP 8.3)	tas-06-02: 10.33.199.51 (IP pública a retirar)	8080/TCP desde 10.33.199.49 · 9117 métricas de Apache	MariaDB, NFS

Servicio / rol	Ubicación	Puerto o flujo	Dependencias
WordPress CMS 2 (contenedor Apache + PHP 8.3)	tas-06-cms-02: 10.33.199.50	8080/TCP desde 10.33.199.49 · 9117 métricas de Apache	MariaDB, NFS
MariaDB (contenedor) + servidor NFS	VM de base de datos: 10.33.199.53	3306/TCP solo desde .50, .51 y .52 · 2049/TCP (NFS) solo desde .50 y .51	CMS 1, CMS 2, monitor
Monitorización (Prometheus, Alertmanager, Loki, Grafana)	tas-06-monitor: 10.33.199.52 (IP pública temporal)	9090, 9093, 3100 y 3000 solo en 127.0.0.1; acceso por túnel SSH	Exporters de los demás nodos
Administración y hora (SSH, NTP)	Todos los nodos	22/TCP desde la red de administración · 123/UDP NTP	Administrador, fuente NTP

4 CONSIDERACIONES DE RECURSOS Y ESCALABILIDAD

Componente	CPU / RAM	Almacenamiento	Supuesto de carga	Evolución
Nodo perimetral (DNS + Nginx)	1 vCPU / 1 GB RAM (estimado)	10 GB (estimado)	Bajo uso de CPU; concentra DNS, TLS y balanceo en un solo host	Separar DNS y Nginx y evaluar un segundo proxy con VRRP
WordPress CMS 1	2 vCPU / 2 GB RAM (estimado)	20 GB (estimado)	Debe poder atender el 100 % del pico previsto (criterio N-1)	Retirar la IP pública; agregar un tercer backend si el pico supera la capacidad medida
WordPress CMS 2	2 vCPU / 2 GB RAM (estimado)	20 GB (estimado)	Igual que CMS 1: también debe absorber el 100 % del pico	Mantener como réplica equivalente de CMS 1
MariaDB + servidor NFS (VM de BD)	2 vCPU / 4 GB RAM (estimado)	Por medir (BD + archivos subidos)	Concentra las consultas de ambos backends y el I/O de los archivos compartidos	Separar el NFS en un nodo propio y replicar la base de datos
Monitorización	2 vCPU / 4 GB RAM (real, con 4 GB de swap)	191,6 GB (real)	Retención de métricas y de logs de 7 días	Ampliar la retención, agregar el agente de logs y un exporter de BIND9

Componente	CPU / RAM	Almacenamiento	Supuesto de carga	Evolución
Respaldos (pendiente)	Por definir	Por definir; retención ≥ 30 días	Hoy solo existe un respaldo manual de la base anterior (octavio_bd_backup.sql)	Backups automatizados de BD, archivos y configuración, con copia fuera del host y restauración probada

Objetivos de servicio: la siguiente tabla fija las metas y la forma de verificarlas. La disponibilidad se mide para el servicio completo, con los puntos únicos de falla aceptados.

Indicador	Meta	Justificación	Verificación
Disponibilidad mensual	≥ 99,5 %	Equivale a unas 3,6 h de indisponibilidad al mes (0,5 % de 720 h)	Disponibilidad medida desde la monitorización
Detección de falla crítica	≤ 5 min	Evita repetir el incidente de casi 2 h sin diagnóstico	Alerta recibida tras detener un servicio, con marcas de tiempo
RTO servicio web	≤ 60 min	Una campaña no puede quedar caída más de 1 h sin impacto comercial	Restauración medida en un ejercicio controlado (Entrega 4)
RPO general	≤ 24 h	Aceptable en operación normal; se mejora en campañas con dumps y binlogs más frecuentes de la BD	Datos creados antes del backup y comprobados tras la restauración
Retención de respaldos	≥ 30 días	Cubre detección tardía de errores, con al menos una copia fuera del host de origen	Inventario de respaldos y checksum

Tolerancia a fallas web frente a disponibilidad del servicio: tener dos backends solo protege contra la caída de uno de ellos, y solo si el otro tiene capacidad suficiente. La disponibilidad del servicio completo depende además del nodo perimetral y de la VM de la base de datos (que aloja MariaDB y el NFS), que son puntos únicos de falla. Con un RTO de 60 min, la falla de cualquiera de ellos cabe en el presupuesto mensual de 3,6 h, pero dos fallas en el mismo mes lo agotan. Por eso este informe habla de tolerancia a fallas en la capa web y no de alta disponibilidad del servicio completo.

Capacidad con un solo backend (criterio N-1): dos nodos no garantizan continuidad si uno no puede absorber la demanda del otro. El criterio de diseño es que cada backend atienda el 100 % del pico previsto. Se medirá con una prueba de carga (k6 o ab) sobre el catálogo y el checkout con un solo backend activo, con umbrales de p95 ≤ 2 s, CPU ≤ 85 % y errores < 1 %, y con la condición de que el pico proyectado de campaña sea menor o igual a la capacidad medida de un backend.

Cálculo de referencia (supuestos ilustrativos que se reemplazarán con las mediciones): con 2 GB de RAM y unos 600 MB reservados para el sistema y Docker, a unos 60 MB por proceso de Apache con PHP caben cerca de 24 procesos, por lo que MaxRequestWorkers debe ajustarse a unos 20, ya que el valor por defecto de la imagen (150) agotaría la memoria. Con 0,3 s por solicitud, la capacidad teórica es de unas 65 solicitudes por segundo; si con plugins reales se alcanza el 50 %, la capacidad estimada es de unas 30 solicitudes por

segundo por backend, y MariaDB debe admitir al menos 2 × 20 conexiones más las del monitor y un margen. Si el pico proyectado supera esa capacidad, se aplican medidas de degradación controlada: caché de página, limit_req en Nginx para proteger el checkout y suspensión de tareas no críticas.

Puntos únicos de falla identificados: el nodo perimetral, que concentra DNS, TLS y balanceo en un mismo host; la VM de la base de datos, donde MariaDB y el servidor NFS comparten host, de modo que una falla elimina datos y archivos a la vez; la VM de monitorización, única, cuya caída deja sin observabilidad pero no sin servicio; y la infraestructura de virtualización y red del laboratorio, compartida por todos los nodos.

Plan de crecimiento: separar DNS y Nginx y evaluar un segundo nodo Nginx con VRRP/keepalived; separar el servidor NFS de la base de datos y replicar MariaDB, con respaldo frecuente de binlogs para mejorar el RPO en campañas; evolucionar los archivos hacia almacenamiento de objetos compatible con S3 si el volumen crece; y agregar un tercer backend si el pico proyectado supera la capacidad medida de un nodo. La subred /29 tiene seis direcciones utilizables y cinco están asignadas; queda libre la .54, por lo que cualquier crecimiento adicional requiere una subred mayor. El balanceo de carga no equivale a alta disponibilidad: esta requeriría además separar DNS de Nginx, replicar la base de datos y garantizar el almacenamiento compartido.

5 USABILIDAD

Los clientes finales acceden a la plataforma mediante el dominio interno tas-06.arpa, sin necesidad de recordar direcciones IP ni distinguir qué backend los atiende. WordPress ofrece una navegación familiar de catálogo, carrito y confirmación de pedidos o reservas, con mensajes de error estándar y consistentes entre ambos backends.

El personal de Nodo Sur, sin formación técnica avanzada, administra la plataforma desde el panel de administración de WordPress, donde puede publicar contenido, gestionar el catálogo y revisar pedidos o reservas, sin necesidad de acceder a la infraestructura subyacente para sus tareas cotidianas.

Las tareas administrativas de infraestructura quedan reservadas al equipo del proyecto mediante acceso SSH nominativo por clave pública hacia las IP privadas de cada nodo, evitando que el personal de negocio necesite privilegios elevados para operar el sitio.

La coherencia del catálogo entre backends se resuelve con el almacenamiento compartido por NFS: una imagen subida desde un backend es visible desde el otro, por lo que el cliente no percibe diferencias entre ser atendido por CMS 1 o por CMS 2. Como el servidor NFS es único, su caída dejaría sin imágenes de catálogo a ambos backends, riesgo declarado en las consideraciones de escalabilidad.

6 RECOMENDACIONES DE SEGURIDAD Y MONITOREO

Riesgo	Control recomendado	Métrica / log	Validación futura
Credenciales compartidas, expuestas o sin revocación individual	SSH nominativo por clave pública (PasswordAuthentication no, PermitRootLogin no) y sudo limitado; secretos de MariaDB, Grafana y de los exporters en archivos .env o .my.cnf con permisos 600, fuera del repositorio; rotación inmediata si se exponen	Logs de autenticación SSH (auth.log)	Entrega 2: revocar una clave de prueba y buscar secretos en el repositorio con git grep
Base de datos expuesta a redes no autorizadas	Publicar el 3306 solo en la IP privada (DB_BIND_IP) y restringir el origen a .50, .51 y .52 en la cadena DOCKER-USER	Logs de MariaDB (Access denied) y reglas de firewall aplicadas	Entrega 2/3: prueba de conexión bloqueada desde un origen no autorizado
Acceso directo a un backend que elude el punto de entrada (hoy, puerto 8080 en todas las interfaces y CMS 1 con IP pública)	Publicar el 8080 solo en la IP privada, aceptar solo 10.33.199.49 y retirar la IP pública del CMS 1	Intentos de acceso directo en los logs del backend y reglas de firewall	Entrega 2: curl a la IP pública desde fuera debe fallar; curl al 8080 desde un origen distinto de .49 debe ser rechazado
Divergencia de archivos entre CMS 1 y CMS 2	NFS en la VM de BD exportado solo a .50 y .51 (sin no_root_squash), con uploads y wp-config.php compartidos; edición de archivos desde el panel deshabilitada (ADR-003)	sha256sum de un mismo archivo servido por ambos backends	Entrega 3: subir un archivo y descargarlo por ambos backends con checksum idéntico en ≤ 5 s
Falla no detectada a tiempo (incidente de 2 horas sin diagnóstico)	Prometheus y Alertmanager con reglas sobre up, nginx_up, apache_up y mysql_up; definir el destino de las alertas; script de comprobación activa por backend que compense la detección pasiva de Nginx libre (ADR-002)	Métricas de disponibilidad, latencia y errores	Entrega 4: degradación controlada y alerta recibida en ≤ 5 min
Backup nunca probado o restauración fallida	Backups automatizados de BD, archivos (NFS) y configuración, con retención ≥ 30 días y una copia fuera del host de origen; dump previo a cualquier operación destructiva	Inventario de respaldos y checksum de verificación	Entrega 4: restauración controlada con medición real de RPO y RTO

Riesgo	Control recomendado	Métrica / log	Validación futura
Un backend no absorbe el pico cuando cae el otro	Criterio N-1 con prueba de carga con un solo backend; ajuste de MaxRequestWorkers, caché y limit_req	p95, CPU y tasa de errores durante la prueba	Entrega 3: con un backend activo se cumple p95 ≤ 2 s, CPU ≤ 85 % y errores < 1 %
Certificado TLS vencido o mal configurado	Certificado validado y renovación gestionada en el nodo perimetral	Logs de Nginx; verificación con curl y openssl	Entrega 2/3: validación de HTTPS desde un cliente externo
Plugins desactualizados o vulnerables	Política de actualización en entorno de pruebas, plugins mínimos, imágenes con etiqueta fija y edición de archivos deshabilitada	Versiones instaladas y avisos de seguridad	Entrega 2/3: inventario de plugins con versión y revisión periódica
Endpoints de estado y métricas expuestos (/server-status de Apache, exporters, estadísticas de BIND9)	Restringir /server-status a la red interna de Docker, publicar los exporters solo en la IP privada y permitir el origen 10.33.199.52; canal de BIND9 solo en 127.0.0.1	Reglas de firewall y logs de acceso	Entrega 2/3: curl desde un origen no autorizado a /server-status y a los puertos 9113 y 9117 debe fallar
Resolvedor DNS abierto (recursión habilitada para cualquier origen)	Limitar allow-recursion y allow-query-cache a la red 10.33.199.48/29 y a localhost; la zona tas-06.arpa sigue respondiendo a todos	Logs de consultas de BIND9	Entrega 2: dig @10.33.195.184 google.com desde fuera debe responder REFUSED

Acceso y segmentación: SSH por clave pública hacia las IP privadas de cada nodo, con un usuario nominativo por integrante. El firewall aplica una política por defecto de denegar: solo el nodo perimetral expone los puertos 53, 80 y 443 hacia la red pública; los backends aceptan el 8080 solo desde .49, la base de datos el 3306 solo desde .50, .51 y .52, y el NFS el 2049 solo desde .50 y .51. Prometheus, Alertmanager, Loki y Grafana escuchan solo en 127.0.0.1 y se acceden por túnel SSH (AllowTcpForwarding local). Los secretos se gestionan fuera del repositorio con archivos .env y .my.cnf de permisos 600, y wp-config.php, que contiene la contraseña de la base de datos, se comparte por NFS con acceso restringido a los backends.

Política de datos: la siguiente tabla define ubicación, respaldo, retención, acceso y capacidad de cada tipo de dato, para evitar la falta de política de capacidad, permisos y retención del incidente piloto.

Dato	Ubicación y respaldo	Retención	Acceso	Capacidad y permisos
Pedidos, reservas, inscripciones y datos de clientes	MariaDB en la VM de BD (10.33.199.53); dump diario y copia separada	≥ 30 días	Solo el usuario de la aplicación desde .50 y .51	Alerta al 80 % de disco
Archivos subidos y wp-config.php	NFS en la VM de BD (/srv/nfs/wordpress); copia diaria	≥ 30 días	Solo .50 y .51; propietario www-data (uid 33), nunca chmod 777	Límite de tamaño por archivo; alerta al 80 % de disco

Dato	Ubicación y respaldo	Retención	Acceso	Capacidad y permisos
Logs	Cada nodo y Loki en el monitor	7 días en Loki (configurado); a definir en cada nodo	Solo administradores nominativos, por túnel SSH	logrotate y límite de tamaño de logs de Docker
Métricas	Prometheus en el monitor (volumen propio)	7 días (configurado)	Solo administradores nominativos, por túnel SSH	Vigilar el uso del volumen
Configuración	Repositorio Git (saneada)	Historial completo	Cuentas nominativas	Sin secretos en el repositorio

Los datos de clientes son datos personales: se debe verificar con el docente la normativa chilena aplicable (Ley 19.628 y su actualización, Ley 21.719) y no incluir datos personales en el repositorio ni en capturas.

7 PLAN DE IMPLEMENTACIÓN, PRUEBAS Y REVERSIÓN

Los responsables son propuestos según la distribución actual del trabajo y deben confirmarse en equipo. Cada etapa avanza solo si se cumple su criterio y cuenta con un procedimiento de reversión.

Etapa y entregable	Responsables y dependencias	Pruebas previstas	Criterio para avanzar	Reversión
Entrega 2: Docker en los nodos, MariaDB y WordPress en contenedores, DNS y Nginx, servidor NFS y VM de monitorización; retiro de las IP públicas	Octavio (CMS), Lukas (BD y NFS), Josecarlos (DNS y nodo perimetral). Depende de VMs y red listas	Resolución DNS de tas-06.arpa, acceso HTTPS por nombre, flujo de compra o reserva, BD inaccesible desde fuera y NFS montado en ambos backends	RF-01, RNF-02 y RNF-07 verificados	Dump de la BD y copia de los compose y de nginx.conf antes de cada cambio; nginx -t antes de reload
Entrega 3: balanceo verificado entre ambos backends, script de salud, firewall en DOCKER-USER y prueba de capacidad N-1	Octavio, Lukas, Josecarlos. Depende de la Entrega 2 y del ADR-003	Distribución entre CMS 1 y CMS 2, retiro de un backend, checksum entre nodos y prueba de carga con un solo backend	RNF-01 y RNF-03 verificados y capacidad de un backend ≥ pico proyectado	Quitar el segundo backend del upstream y volver a un solo nodo

Etapa y entregable	Responsables y dependencias	Pruebas previstas	Criterio para avanzar	Reversión
Entrega 4: alertas con destino, exporter de BIND9, agente de logs hacia Loki, backups automatizados y restauración	Equipo completo. Depende de la Entrega 3	Degradación controlada, alerta ≤ 5 min y restauración ≤ 60 min en una VM de prueba	RNF-05, RNF-06 y RNF-08 verificados con marcas de tiempo	Restaurar la configuración anterior desde Git
Entrega 5: auditoría final e incidente no anunciado	Equipo completo. Depende de la Entrega 4	Recuperación validada con el flujo de negocio y defensa individual	Matriz de pruebas completa	Etiqueta de versión estable del repositorio

8 CONCLUSIONES

La propuesta es viable dentro de las restricciones del caso: se implementó con software libre (WordPress, Nginx, MariaDB, Prometheus y Grafana) sobre contenedores y puede ser administrada por personal sin formación técnica avanzada. La arquitectura vigente ya cuenta con dos backends equivalentes, base de datos separada, archivos compartidos por NFS y monitorización; faltan por cerrar las alertas con destino, el envío de logs, los respaldos automatizados y la restricción del acceso directo a los backends.

Las decisiones prioritarias son: restringir el acceso directo a los backends y retirar la IP pública del CMS 1; definir el destino de las alertas de Alertmanager; automatizar los respaldos y probar su restauración; limitar la recursión de BIND9; medir la capacidad de un backend para validar el criterio N-1 y ajustar MaxRequestWorkers; y confirmar el plugin de reservas con cupos. El dimensionamiento de las VM es hoy una estimación, salvo el de la VM de monitorización, y debe sustituirse por mediciones.

Los principales riesgos residuales son los puntos únicos de falla: el nodo perimetral que concentra DNS, TLS y Nginx, y la VM de la base de datos que aloja MariaDB y el NFS. El diseño entrega tolerancia a fallas en la capa web y no alta disponibilidad del servicio completo; su evolución queda como trabajo futuro, más allá del alcance de esta entrega.